

Blog Website Platform

Project 01

1. Product Overview

This document defines the requirements for a full-stack Blog Website Platform. The platform enables users to register, authenticate, and manage blog content — including creating, reading, updating, and deleting posts — backed by a persistent database.

2. Goals & Objectives

- Build a fully functional blog platform with user authentication and CRUD operations.
- Deliver a responsive, intuitive UI accessible on both desktop and mobile.
- Establish a scalable backend architecture using Node.js and a document/NoSQL database.
- Provide a foundation for future feature extensions (comments, categories, analytics).

3. Scope

3.1 In Scope

- User registration, login, and logout
- Authenticated blog post creation, editing, and deletion
- Public blog post listing and detail view
- Frontend SPA using HTML/CSS/JavaScript or React
- Backend REST API using Node.js
- Persistent data storage with MongoDB or Firebase

3.2 Out of Scope

- Social login (OAuth with Google/Facebook)
- Comments and user reactions (Phase 2)
- Analytics dashboard
- Email notifications
- Multi-language / i18n support

4. Technical Stack

Feature	Priority	Description
Frontend	Required	HTML5, CSS3, JavaScript (ES6+) or React
Backend	Required	Node.js with Express.js REST API
Database	Required	MongoDB (Mongoose) or Firebase Firestore
Auth	Required	JWT-based session management
Hosting	Optional	Vercel / Render / Firebase Hosting

5. User Roles & Personas

Guest User

Unauthenticated visitor who can browse and read published blog posts but cannot create or manage content.

Registered Author

Authenticated user who can create, edit, and delete their own blog posts via the dashboard.

6. Functional Requirements

6.1 Authentication

Feature	Priority	Description
User Registration	P0	Form with username, email, password. Password hashed before storage.
User Login	P0	Email + password login. JWT issued on success.
User Logout	P0	Invalidate token / clear session on client.
Protected Routes	P0	Create/edit/delete endpoints require valid JWT.
Input Validation	P1	Client- and server-side validation with error messages.

6.2 Blog Post Management (CRUD)

Feature	Priority	Description
Create Post	P0	Title, body (rich text or markdown), tags, publish date.
Read / List Posts	P0	Public listing page with pagination (10 posts/page).
Read Single Post	P0	Detail page with full content, author, and date.
Update Post	P0	Author can edit their own posts; changes persist immediately.

Feature	Priority	Description
Delete Post	P0	Author can delete their own posts with a confirmation prompt.
Draft Support	P1	Save posts as drafts; only published posts visible publicly.

6.3 UI / UX

Feature	Priority	Description
Responsive Layout	P0	Desktop and mobile-friendly using CSS Grid/Flexbox.
Navigation	P0	Header with logo, nav links, auth state (Login / Profile).
Blog Card Grid	P0	All Posts view with image, author, date, and tags per card.
Post Detail Page	P0	Full post with author info, tags, and back navigation.
Empty State	P1	Friendly placeholder when no posts exist.
Loading States	P1	Spinners/skeletons during API calls.

7. Non-Functional Requirements

- Performance: API responses under 500ms for standard CRUD operations.
- Security: Passwords hashed with bcrypt (min cost factor 10). JWTs expire within 24 hours.
- Scalability: Database schema designed to support 10,000+ posts without restructuring.
- Code Quality: Modular folder structure; linting enforced (ESLint).
- Browser Support: Chrome, Firefox, Safari, Edge — latest two major versions.

8. Data Models

8.1 User

Feature	Priority	Description
_id	String/ObjectId	Unique identifier (auto-generated)
username	String	Unique display name
email	String	Unique email address
passwordHash	String	bcrypt hashed password
createdAt	Date	Account creation timestamp

8.2 Blog Post

Feature	Priority	Description
_id	String/ObjectId	Unique identifier
title	String	Post headline (max 200 chars)
body	String	Full post content (Markdown or HTML)
authorId	Reference	FK → User._id
tags	Array<String>	Category tags (e.g. Design, Research)
status	Enum	draft published
publishedAt	Date	Publish date/time
updatedAt	Date	Last edit timestamp

9. API Endpoints

Feature	Priority	Description
POST /api/auth/register	Public	Register a new user account
POST /api/auth/login	Public	Authenticate and receive JWT
POST /api/auth/logout	Auth	Clear user session
GET /api/posts	Public	List all published posts (paginated)
GET /api/posts/:id	Public	Get a single post by ID
POST /api/posts	Auth	Create a new post
PUT /api/posts/:id	Auth	Update an existing post
DELETE /api/posts/:id	Auth	Delete a post (author only)

10. Milestones & Timeline

Feature	Priority	Description
M1 – Environment Setup	Week 1	Project scaffolding, DB connection, folder structure
M2 – Auth System	Week 2	Registration, login, JWT middleware
M3 – Post CRUD API	Week 3	All CRUD endpoints with validation
M4 – Frontend UI	Week 4–5	Responsive blog listing, detail, and dashboard pages
M5 – Integration & QA	Week 6	End-to-end testing, bug fixes, deployment

11. Success Metrics

- All P0 functional requirements passing end-to-end tests.

- Zero critical security vulnerabilities (OWASP Top 10 checked).
 - Page load time < 2 seconds on a standard broadband connection.
 - Mobile Lighthouse score ≥ 80 for Performance and Accessibility.
-